

LAB # 05

EXERCISE:

1. Consider Triangle problem : /* Design and develop a program in a language of your choice to solve the triangle problem defined as follows : Accept three integers which are supposed to be the three sides of triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Derive test cases for your program based on decision-table approach, execute the test cases and discuss the results */ and attached all printouts /screenshots/summary report here.

Conditions:

C1: $a < b + c$?

C2: $b < a + c$?

C3: $c < a + b$?

C4: $a = b$?

C5: $a = c$?

C6: $b = c$?

Actions:

A1: Not a triangle

A2: Scalene

A3: Isosceles

A4: Equilateral

No. Of Columns = $2^{\text{conditions}}$

No. Of Columns = $2^6 = 64$

1. **Make initial decision table consisting of columns (using the formula $2^{\text{conditions}}$)**

A	B	C	D	E	F	G	H	I	J	K
Conditions	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
C1: $a < b + c$?	T	T	T	T	F	T	F	F	F	F
C2: $b < a + c$?	T	T	T	T	F	T	F	F	-	-
C3: $c < a + b$?	T	T	T	T	F	T	F	F	-	-
C4: $a = b$?	T	F	T	F	F	F	T	T	-	-
C5: $a = c$?	T	F	F	T	F	T	F	T	-	-
C6: $b = c$?	T	F	F	F	T	T	T	F	-	-
Actions:										
Not a triangle									T	T
Scalene		T								
Isosceles			T	T	T					
Equilateral	T									
Impossible						T	T	T		

L	M	N	O	P	Q	R	S	T
R11	R12	R13	R14	R15	R16	R17	R18	R19
F	F	F	F	F	F	F	F	F
-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-
T	T	T	T	T	T	T	T	T

[illegible][illegible]

R41	R42	R43	R44	R45	R46	R47	R48	R49	R50	R51	R52
T	T	T	T	T	T	T	T	T	T	T	T
F	F	F	F	F	F	F	F	F	F	F	F
-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-
T	T	T	T	T	T	T	T	T	T	T	T

R52	R53	R54	R55	R56	R57	R58	R59	R60	R61	R62	R63	R64
T	T	T	T	T	T	T	T	T	T	T	T	T
F	F	F	F	F	F	F	F	F	F	F	F	F
-	-	-	-	-	F	F	F	F	F	F	F	F
-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-
T	T	T	T	T	T	T	T	T	T	T	T	T

2. Reduced the table to remove redundant/impossible scenarios.

A	B	C	D	E	F	G	H	I	J	K	L
Conditions	R1	R2	R3	R4	R5	R6	R7	R8	R (9-40)	R (41-56)	R (57-64)
C1: $a < b + c$?	T	T	T	T	F	T	F	F	F	T	T
C2: $b < a + c$?	T	T	T	T	F	T	F	F	-	F	T
C3: $c < a + b$?	T	T	T	T	F	T	F	F	-	-	F
C4: $a = b$?	T	F	T	F	F	F	T	T	-	-	-
C5: $a = c$?	T	F	F	T	F	T	F	T	-	-	-
C6: $b = c$?	T	F	F	F	T	T	T	F	-	-	-
Actions:											
Not a triangle									T	T	T
Scalene		T									
Isosceles			T	T	T						
Equilateral	T										
Impossible						T	T	T			

3. Write the test cases for a reduced decision table.

Test Case ID	a	b	c	Expected Result	Actual Result
1	1	2	3	Not a Triangle	Not a Triangle
2	2	3	5	Not a Triangle	Not a Triangle
3	5	1	2	Not a Triangle	Not a Triangle
4	3	4	5	Scalene	Scalene
5	5	5	3	Isosceles	Isosceles
6	7	7	10	Isosceles	Isosceles
7	-	-	-	Impossible	Impossible
8	6	6	8	Isosceles	Isosceles
9	-	-	-	Impossible	Impossible
10	-	-	-	Impossible	Impossible
11	4	4	4	Equilateral	Equilateral

4. Execute the code for triangle problem and execute test cases

```
#include <iostream>
using namespace std;
bool isValidTriangle(int a, int b, int c) {
    return (a + b > c) && (a + c > b) && (b + c > a);
}
string triangleType(int a, int b, int c) {
    if (!isValidTriangle(a, b, c)) {
        return "Not a Triangle";
    } else if (a == b && b == c) {
        return "Equilateral Triangle";
    } else if (a == b || b == c || a == c) {
        return "Isosceles Triangle";
    } else {
        return "Scalene Triangle";
    }
}
.
```

```
int main() {
    int a, b, c;
    cout << "Enter three sides of the triangle: ";
    cin >> a >> b >> c;
    cout << "Result: " << triangleType(a, b, c) << endl;
    return 0;
}
```

Test Case Execution

```
Enter three sides of the triangle: 1 1 2  
Result: Not a Triangle
```

```
Enter three sides of the triangle: 2 2 3  
Result: Isosceles Triangle
```

```
Enter three sides of the triangle: 3 5 1  
Result: Not a Triangle
```

```
Enter three sides of the triangle: 3 4 5  
Result: Scalene Triangle
```

```
Enter three sides of the triangle: 5 5 3  
Result: Isosceles Triangle
```

```
Enter three sides of the triangle: 7 7 10  
Result: Isosceles Triangle
```

7) Impossible

```
Enter three sides of the triangle: 6 6 8  
Result: Isosceles Triangle
```

9) Impossible

10)Impossible

Enter three sides of the triangle: 4 4 4
Result: Equilateral Triangle

2. Consider a library management system(LMS) which issues book(s) to a registered user by checking the outstanding fee i.e. if a registered user have no pending fee then requested book may be issued(if and only if the potential user has under borrow limit)
 - i. Make initial decision table consisting of columns (using the formula $2^{\text{conditions}}$)
 - ii. Reduced the table to remove redundant/impossible scenarios.
 - iii. write test cases for reduced decision table
 - iv. execute code for LMS problem and execute test cases
 - v. Design and develop a program in a language of your choice to develop above mentioned problem and attached all printouts /screenshots/summary report here.

Make initial decision table consisting of columns (using the formula $2^{\text{conditions}}$)

DECISION TABLE

Conditions	Rule1	Rule2	Rule3	Rule4	Rule5	Rule6	Rule7	Rule8
C1: Registered	T	T	T	T	F	F	F	F
C2: No Pending Fee	T	T	F	F	T	T	F	F
C3: Under borrow limit	T	F	T	F	T	F	T	F
Actions								
Yes	Yes							
No		No	No	No	No	No	No	No

Reduced the table to remove redundant/impossible scenarios.

REDUCED TABLE

Conditions	Rule1	Rule2	Rule3,4	Rule5,6,7,8
C1: Registered	T	T	T	F
C2: No Pending Fee	T	T	F	-
C3: Under borrow limit	T	F	-	-
Actions				
Yes	Yes			
No		No	No	No

write test cases for reduced decision table

Test Case ID	Test Description	C1: Registered	C2: No Pending Fee	C3: Under Borrow Limit	Expected Result
TC1	Registered, no pending fee, under borrow limit	T	T	T	Book Issued
TC2	Registered, no pending fee, exceeded borrow limit	T	T	F	Book Not Issued
TC3	Registered, pending fee, under borrow limit	T	F	yes/no	Book Not Issued
TC4	Not registered	F	yes/no	yes/no	Book Not Issued

Execute code for LMS problem and execute test cases.

Design and develop a program in a language of your choice to develop above mentioned problem and attached all printouts /screenshots/summary report here.

```
class User:
```

```

def __init__(self, user_id, registered, no_pending_fee, borrow_limit):
    self.user_id = user_id
    self.registered = registered
    self.no_pending_fee = no_pending_fee
    self.borrow_limit = borrow_limit

class LibraryManagementSystem:
    def issue_book(self, user):
        """Function to determine if the book can be issued based on conditions."""
        if not user.registered:
            return "Book Not Issued: User not registered"
        if not user.no_pending_fee:
            return "Book Not Issued: Pending fee exists"
        if not user.borrow_limit:
            return "Book Not Issued: Borrow limit exceeded"

        return "Book Issued"

def get_user_input(test_case_id, description):
    """Function to take user input for each test case"""
    print(f"\n ♦ Enter details for {test_case_id}: {description}")

    registered = input("Is the user registered? (yes/no): ").strip().lower() ==
"yes"

    # If the user is not registered, other conditions don't matter
    if not registered:
        return User(test_case_id, registered, False, False)

    no_pending_fee = input("No pending fee? (yes/no): ").strip().lower() ==
"yes"

    borrow_limit = input("Is the user within the borrow limit? (yes/no):
").strip().lower() == "yes"

    return User(test_case_id, registered, no_pending_fee, borrow_limit)

```



```

def main():
    print("\nLibrary Management System - Issue Book\n")

    test_cases_data = [
        ("TC1", "Registered, no pending fee, under borrow limit"),
        ("TC2", "Registered, no pending fee, exceeded borrow limit"),
        ("TC3", "Registered, pending fee, under borrow limit"),
        ("TC4", "Not registered")
    ]

    test_cases = []
    lms = LibraryManagementSystem()

    # Taking input for each test case
    for case_id, description in test_cases_data:
        user = get_user_input(case_id, description)
        result = lms.issue_book(user)
        print(f"Output for {case_id}: {result}\n")
        test_cases.append((case_id, description, user, result))

    # Print test summary report
    print("\n Test Summary Report:")
    print("{:<6} {:<50} {:<10} {:<10} {:<10} {:<30}".format(
        "ID", "Test Description", "C1", "C2", "C3", "Expected Result"))
    print("="*110)

    for case_id, description, user, result in test_cases:
        print("{:<6} {:<50} {:<10} {:<10} {:<10} {:<30}".format(
            case_id, description, str(user.registered), str(user.no_pending_fee),
            str(user.borrow_limit), result))

if __name__ == "__main__":
    main()

```

```
PS D:\PYTHON> python -u "d:\PYTHON\LMS.py"
```

Library Management System - Issue Book

- ◆ Enter details for TC1: Registered, no pending fee, under borrow limit

Is the user registered? (yes/no): yes

No pending fee? (yes/no): yes

Is the user within the borrow limit? (yes/no): yes

Output for TC1: Book Issued

- ◆ Enter details for TC2: Registered, no pending fee, exceeded borrow limit

Is the user registered? (yes/no): yes

No pending fee? (yes/no): yes

Is the user within the borrow limit? (yes/no): no

Output for TC2: Book Not Issued: Borrow limit exceeded

- ◆ Enter details for TC3: Registered, pending fee, under borrow limit

Is the user registered? (yes/no): yes

No pending fee? (yes/no): no

Is the user within the borrow limit? (yes/no): -

Output for TC3: Book Not Issued: Pending fee exists

- ◆ Enter details for TC4: Not registered

Is the user registered? (yes/no): no

Output for TC4: Book Not Issued: User not registered

Test Summary Report:

ID	Test Description	C1	C2	C3	Expected Result
TC1	Registered, no pending fee, under borrow limit	True	True	True	Book Issued
TC2	Registered, no pending fee, exceeded borrow limit	True	True	False	Book Not Issued: Borrow limit exceeded
TC3	Registered, pending fee, under borrow limit	True	False	False	Book Not Issued: Pending fee exists
TC4	Not registered	False	False	False	Book Not Issued: User not registered

```
PS D:\PYTHON> python -u "d:\PYTHON\LMS.py"
```